

SUPSI

Proxy basato su Appwrite

Studente/i

**Mazza Luca
Mondini Manuela**

Relatore

Brocco Amos

Correlatore

-

Committente

Brocco Amos

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11160

Anno

2025 / 2026

Data

30 marzo 2026

Indice

1	Abstract	1
2	Introduzione	3
2.1	Pianificazione	4
2.2	Requisiti	6
3	Stato dell'arte	7
3.1	Limiti delle infrastrutture di rete tradizionali	7
3.2	Tecnologie di superamento dei Firewall	7
3.3	Protocolli di messaggistica e middleware	8
3.4	Il paradigma Backend-as-a-Service (BaaS)	8
3.5	Il ruolo di Appwrite come middleware evoluto	8
3.6	WebAssembly e Qt	9
3.7	Comparativa delle tecnologie	9
3.8	Conclusione	9
4	Design e implementazione	10
4.1	Modello di dominio	10
4.1.1	Struttura del modulo	10
4.1.2	Entità principali	10
4.1.3	Serializzazione e parsing	11
4.1.4	Validazione	11
4.1.5	Gestione della sessione	11
4.1.6	Integrazione con gli altri componenti	11
4.2	Appwrite SDK	11
4.2.1	Componenti principali	12
4.2.2	Comunicazione asincrona (con signals e slots)	12
4.2.3	Gestione della sessione	12
4.2.4	Dualità Server-Client	13
4.2.5	Utilizzo	13
4.2.6	Gestione degli errori	13
4.3	Resilienza e controllo	13
4.3.1	Cache dei messaggi	14
4.3.2	Gestione della recovery	14
4.3.3	Controllo del Throughput	15
4.4	Gestione degli errori	16
4.4.1	Classificazione degli errori	16
4.4.2	Struttura dell'errore	17
4.4.3	Rilevazione e formattazione	17
4.4.4	Integrazione nel sistema	17
4.4.5	Scelte progettuali	17

4.5	Gestione dello stato del client	17
4.5.1	Struttura dello stato	18
4.5.2	Gestione dell'ordinamento e recovery	18
4.5.3	Contesto di comunicazione	18
4.5.4	Stato di autenticazione	18
4.5.5	Stato della connessione	18
4.5.6	Integrazione con gli altri componenti	19
4.5.7	Scelte progettuali	19
4.6	Elaborazione dei messaggi in ingresso	19
4.6.1	Ruolo del componente	19
4.6.2	Dipendenza dallo stato del client	20
4.6.3	Metodo processIncoming()	20
4.6.4	Deduplicazione	20
4.6.5	Rilevazione di gap	20
4.6.6	Aggiornamento dello stato	21
4.6.7	Reset del contesto	21
4.6.8	Scelte progettuali	21
5	Test e validazione	22
5.1	Risultati	22
6	Risultati	23
6.1	Manuale d'uso	23
7	Conclusioni	24
7.1	Sviluppi futuri	24
	Glossario	I
	Sitografia	III

Elenco delle figure

4.1	Struttura di classi di AppwriteSDK	12
4.2	Struttura di RecentMessageCache	14
4.3	Struttura di RecoveryManager	15
4.4	Struttura di RateLimiter	16

Elenco delle tabelle

2.1	Tecnologie utilizzate nel progetto	4
2.2	Requisiti di Appcomm	6
3.1	Analisi comparativa delle tecnologie	9
4.1	Codici di errore	16
4.2	Stati di connessione	19

Capitolo 1

Abstract

Le policy di sicurezza di rete moderne spesso isolano sia client che server rendendo impraticabili le connessioni dirette senza l'utilizzo di complesse configurazioni. Questo progetto espone il design e lo sviluppo di un'architettura client-server, pensata per superare le limitazioni imposte da firewall e NAT nelle reti private. Il meccanismo centrale sostituisce le tradizionali connessioni socket con un approccio basato su REST + WebSocket, utilizzando Appwrite¹ [1] come Proxy. Il sistema crea un canale di comunicazione sicuro su cui client e server interagiscono asincronamente tramite eventi di creazione di documenti nel database di Appwrite.

Viene quindi sviluppata una libreria modulare in C++ (con QT [2]) per astrarre l'API di Appwrite, implementando funzionalità come la gestione delle sessioni, l'identificazione dei canali basata su UUID e la serializzazione dei payload in JSON. L'implementazione di tale libreria viene validata attraverso lo sviluppo di una semplice applicazione Proof of Concept (PoC), un frontend, realizzato con Qt/QML e compilato con WebAssembly [3] per l'esecuzione in browser, e un servizio backend nativo in C++.

Modern security policies often restrict direct client-server communication, especially in private networks. This project details the design and development of a client-server architecture, designed to overcome the limitations of firewalls and NAT in private networks. The core mechanism replaces the traditional client-server socket connections with a REST + WebSocket based approach, using Appwrite as a Proxy. The system creates a secure communication channel on which clients and servers interact asynchronously via document creation events in the Appwrite-hosted database.

A modular C++ (with QT) library is developed to abstract the Appwrite API, implementing features such as session management, UUID-based channel identification and JSON payload serialization. The implementation of said library is validated through the development of a simple Proof of Concept (PoC) application, a frontend, built with Qt/QML and compiled with WebAssembly for browser-based execution, and a native C++ backend service.

¹<https://appwrite.io>

Capitolo 2

Introduzione

Il progetto nasce dall'esigenza di risolvere una delle sfide che più comunemente affligge le architetture distribuite moderne, ovvero la comunicazione bidirezionale asincrona tra un client ed un server che risiedono in reti private.

Lo scopo del progetto è quindi lo sviluppo di una libreria C++ che permetta di utilizzare Appwrite come middleware (proxy) per facilitare la creazione di canali di comunicazione sicuri e scalabili tra client e server. Al posto di tentare connessioni dirette instabili o poco sicure, entrambe le entità si connettono ad un'istanza Appwrite che funge da intermediario autenticato per lo scambio di messaggi strutturati in tempo reale. Il sistema deve garantire la sicurezza, fornendo accesso solamente ad utenti autenticati e autorizzati per l'accesso ai canali di comunicazione, delegando la gestione di credenziali, sicurezza e persistenza all'infrastruttura sottostante di Appwrite.

Gli obiettivi principali del lavoro sono articolati in diverse aree chiave:

- **Progettazione di un architettura sicura:** è necessario definire un modello di comunicazione dove il server agisca da amministratore dell'infrastruttura di comunicazione (essendo autenticato tramite API Key) ed il frontend come utente finale (autenticandosi come utente "normale"). Il server è inoltre responsabile della gestione dei canali di comunicazione, fornendo un meccanismo di creazione, lettura, aggiornamento e cancellazione (CRUD) per canali, sessioni, mentre invece il frontend si preoccupa semplicemente di connettersi a canali esistenti, inviare messaggi e ricevere notifiche di nuovi messaggi.
- **Sviluppo della libreria appcomm:** una libreria modulare, scritta in C++ e basata su Qt, che fornisca un'interfaccia semplice ed intuitiva per interagire con l'API di Appwrite è necessaria per la strutturazione del progetto; questa deve includere funzionalità per la gestione delle sessioni, l'identificazione dei canali basandosi su UUID, la serializzazione, validazione e impacchettamento dei payload e la gestione degli eventi di comunicazione in modo asincrono.
- **Proof of Concept:** per validare l'efficacia e il funzionamento corretto del sistema è necessario lo sviluppo di una semplice applicazione che dimostri la maniera di utilizzo della libreria in un contesto reale ed il funzionamento tecnico del meccanismo di comunicazione. Questa consiste in una chat semplice, dove un frontend, costruito con Qt/QML e compilato con WebAssembly per l'esecuzione tramite web browser, comunica con un servizio backend nativo in C++, attraverso i canali di comunicazione gestiti da Appwrite tramite appcomm. Per semplificare la dimostrazione, il backend si limita al ruolo di *echo server*.
- **Bootstrap deploy:** per facilitare l'adozione dell'intermediario Appwrite in soluzioni esistenti, è necessario che, in presenza di un'istanza "pulita" di Appwrite ed il server, quest'ultimo sia in grado, solamente tramite la chiave API e l'URL dell'istanza, di creare in

autonomia l'infrastruttura necessaria per il funzionamento, dimodoché sia possibile integrare Appwrite sia come "primo cambiamento" in un'architettura esistente, sia nel caso si necessiti di scalare orizzontalmente un sistema già configurato con questo proxy, permettendo di cambiare istanza in modo dinamico, senza necessità di riconfigurazione.

Il prodotto si avvale di uno stack tecnologico moderno e cross-platform, con l'obiettivo di massimizzare la portabilità e l'integrazione in diversi contesti applicativi, con un focus particolare su soluzioni web-based grazie alla compilazione in WebAssembly.

Tecnologia	Ruolo
Appwrite	Funge da middleware (proxy) per la comunicazione tra client e server
C++	Linguaggio di programmazione per la libreria e il backend
Qt	Framework per lo sviluppo della libreria e del frontend
QML	Linguaggio per la definizione dell'interfaccia utente del frontend
WebAssembly	Compilazione del frontend per l'esecuzione in browser
JSON	Formato per la serializzazione dei messaggi scambiati tra client e server
Docker	Contenitore per il deploy dell'istanza Appwrite e del backend

Tabella 2.1: Tecnologie utilizzate nel progetto

2.1 Pianificazione

Di seguito sono descritte le fasi principali che sono state pianificate per lo sviluppo del progetto. Il piano non è ovviamente rigido, data la natura iterativa dello sviluppo software, ma fornisce una roadmap generale per l'implementazione e la validazione del sistema.

La presenza di un piano che non sia troppo statico è fondamentale per valorizzare l'obiettivo di sviluppo piuttosto che la semplice realizzazione di un prodotto, permettendo di adeguarsi dinamicamente alle esigenze che potenzialmente emergono durante le varie fasi di sviluppo e di discussione con il relatore.

Essendo anche le tecnologie usate relativamente nuove per tutti i membri del team, soprattutto per quanto riguarda Appwrite e WebAssembly, è stato necessario prevedere una fase iniziale di analisi e studio approfondito, prima di procedere con l'implementazione vera e propria, per garantire una comprensione solida dei meccanismi da implementare.

1. **Analisi:** occorre dello studio dedicato ad approfondire come possa essere utilizzato Appwrite come intermediario per la comunicazione tra client e server, comprendere come adattare l'API dello stesso alle esigenze tecniche del progetto e studiare le tecnologie coinvolte;
2. **Pianificazione:** è necessario definire un piano di sviluppo per quanto riguarda la libreria appcomm, identificando le funzionalità chiave da implementare, formalizzandole tramite delle task. Questo va ripetuto anche per la parte di sviluppo del frontend e del backend;
3. **Design:** è necessario definire un'architettura per la libreria appcomm che sia modulare, scalabile e manutenibile, identificando i moduli chiave, le loro responsabilità e le interazioni tra di essi. Inoltre, è necessario definire un'architettura generale per il sistema, identificando i componenti chiave, le loro responsabilità e le interazioni tra di essi. Ciò include la produzione di diversi diagrammi che al meglio esplichino le scelte architettureali, come ad esempio diagrammi delle classi, diagrammi di sequenza e diagrammi di flusso;

4. **Implementazione Libreria:** occorre implementare dapprima la libreria appcomm, la quale verrà implementata con il classico approccio per task, dove si analizza dapprima il relativo requisito, si implementa quanto necessario per soddisfarlo, si definisce un'unità di test per validarne il funzionamento e si procede con quanto segue. Lo sviluppo della libreria deve essere imperativamente preceduto da una fase di design dell'architettura della stessa, per garantire una struttura modulare, scalabile e manutenibile;
5. **Implementazione PoC:** una volta che la libreria è sufficientemente matura, è possibile procedere con lo sviluppo del Proof of Concept, il quale ha l'unico scopo di validare il funzionamento tecnico del sistema, deve quindi essere il più semplice possibile, limitato appunto alla dimostrazione del funzionamento della libreria;
6. **Test e Validazione:** una volta che il PoC è completo, è necessario procedere con una fase di test e validazione del prodotto finito, per garantire che tutte le funzionalità siano implementate correttamente e che il sistema funzioni come previsto, dal punto di vista del client e da quello del server, in scenari di comunicazione reali;
7. **Documentazione:** parallelamente alle fasi di sviluppo, è necessario procedere con la stesura della documentazione tecnica del progetto, che include sia la documentazione della libreria appcomm, sia un manuale d'uso per il prodotto finito, che spieghi come utilizzare il sistema, come configurarlo e come integrarlo in soluzioni esistenti. Questa guida utente riassume la documentazione tecnica integrata nei vari moduli della libreria, fornendo esempi pratici di utilizzo e consigli di troubleshooting per facilitare l'adozione del sistema.

Questo rapporto è stato scritto durante tutta la durata del progetto, parallelamente a tutte le fasi descritte sopra.

2.2 Requisiti

ID	Descrizione	Note
R-01	<i>Appcomm</i> deve astrarre l'API di Appwrite	Modulo dedicato
R-02	<i>Appcomm</i> deve esporre lo stato di connessione con Appwrite	-
R-03	<i>Appcomm</i> detiene dei modelli di dominio	-
R-04	<i>Appcomm</i> deve filtrare i messaggi sulla base del destinatario	-
R-05	<i>Appcomm</i> necessita un sistema di Recovery per i messaggi persi	-
R-06	<i>Appcomm</i> tiene una cache degli ultimi n messaggi	-
R-07	<i>Appcomm</i> immagazzina una policy di persistenza dei messaggi	-
R-08	<i>Appcomm</i> gestisce un modello unificato per gli errori	-
R-09	<i>Appcomm</i> rende intercambiabile la modalità di comunicazione (RT/-Poll)	-
R-10	<i>Appcomm</i> fornisce un Wrapper per la gestione di connessione/eventi	-

Tabella 2.2: Requisiti di Appcomm

Capitolo 3

Stato dell'arte

L'interconnessione tra sistemi residenti in reti locali protette e interfacce web rappresenta una delle sfide architettoniche più rilevanti nei sistemi distribuiti moderni. Il problema fondamentale risiede nella natura asimmetrica dei sistemi NAT e Firewall, che permettono, tipicamente senza particolari filtri, le connessioni Outbound ma bloccano sistematicamente i tentativi di accesso Inbound.

3.1 Limiti delle infrastrutture di rete tradizionali

La maggior parte delle reti locali opera dietro configurazioni NAT e Firewall Stateful. Mentre queste tecnologie proteggono la rete interna, impediscono l'instaurazione di socket passivi all'interno della LAN, che siano raggiungibili dall'esterno.

Le soluzioni storiche, come il Port Forwarding o UPnP sono oggi considerate deprecate in contesti professionali a causa dell'elevata vulnerabilità ai vettori d'attacco *Zero-day* e della dipendenza verso hardware di rete, che non garantisce la portabilità della soluzione.

3.2 Tecnologie di superamento dei Firewall

Attualmente, le metodologie per consentire la comunicazione tra entità separate da barriere di rete si dividono in tre categorie principali, che rappresentano le soluzioni più *tradizionali*; ognuna di queste ha dei limiti operativi specifici:

- **Port Forwarding & Reverse Proxy:** Queste tecnologie prevedono l'apertura statica di porte direttamente sui router, oppure di esporre alla rete WAN un server che risiede invece nella LAN. Sebbene possano essere efficienti, aumentano la superficie di attacco del backend e richiedono privilegi di amministrazione sulla rete non sempre disponibili in contesti aziendali o sotto reti mobili.
- **VPN (Virtual Private Network):** Consiste nella creazione di un tunnel logico attraverso la rete, nel quale è possibile comunicare direttamente in maniera sicura. Queste tuttavia introducono un importante overhead in termini di latenza, e spesso richiedono l'installazione di software specifici per il client, rendendole inadatte per applicazioni web-based leggere o temporanee.
- **Protocolli Pub/Sub (MQTT¹/AMQP²):** Utilizzano un broker centrale come punto di incontro. Sebbene ideali per l'IoT, la loro integrazione in ambienti Web puro richiede

¹<https://mqtt.org/>

²<https://amqp.org/>

spesso gateway WebSocket aggiuntivi, complicando il deployment in architetture cross-platform.

3.3 Protocolli di messaggistica e middleware

Esistono diverse tecnologie che tentano di risolvere il problema del *punto di incontro* tra due entità separate.

Tra queste appare REST, il quale è de-facto lo standard per il web. Tuttavia REST è per natura *stateless*, ed è basato su un modello request-response. Per ottenere la bidirezionalità necessaria ad un sistema di controllo o ad una chat, richiede tecniche di Long Polling³, che risultano inefficienti in termini di banda e latenza.

Un'altra tecnologia è WebRTC, il quale è un protocollo avanzato, mirato alla comunicazione Peer to Peer. Sebbene offra latenze minime, la sua implementazione in C++ nativo richiede librerie di enormi dimensioni ed è estremamente complessa da integrare in un sistema Wasm senza un server di segnalazione esterno.

Infine è anche presente MQTT, il quale è invece un protocollo leggero di tipo Pub/Sub. È molto efficace, ma la sicurezza è spesso delegata a strati esterni, e non offre una persistenza dei dati strutturata.

3.4 Il paradigma Backend-as-a-Service (BaaS)

L'evoluzione dei servizi Cloud ha portato all'emergere di piattaforme BaaS⁴ (come Firebase, Supabase o Appwrite). Queste soluzioni agiscono come un middleware neutro sulla rete pubblica. Il vantaggio risiede nel fatto che sia il client che il server effettuano connessioni outbound verso la piattaforma, che sono permesse dai firewall standard, eliminando la necessità di esporre porte fisiche sul backend nativo.

3.5 Il ruolo di Appwrite come middleware evoluto

Appwrite si inserisce in questo scenario non solo come un database, ma come un Orchestratore di Backend. A differenza di un semplice broker di messaggi, Appwrite fornisce:

- **Identity Management:** L'integrazione nativa di sessioni utente (JWT o Cookie) permette di proteggere i canali di comunicazione senza dover implementare logiche di cifratura custom.
- **Protocollo Realtime:** Appwrite astrae la gestione dei socket. Il client si sottoscrive a un canale (documento) e riceve notifiche push al variare dello stato, simulando una comunicazione diretta pur passando per un proxy sicuro.
- **Self-Hosting/Cloud:** Seguendo la filosofia dell'indipendenza tecnologica, il supporto a Docker permette di evitare il Vendor Lock-in, tipico di piattaforme come Firebase.

³<https://ably.com/topic/long-polling>

⁴<https://geeksforgeeks.org/firebase/what-is-firebase/>

3.6 WebAssembly e Qt

L'aspetto che porta l'innovazione più grande del progetto è la possibilità di utilizzare WebAssembly per compilare applicazioni Qt, per poi essere in grado di utilizzarle tramite Web Browser. Ciò permette di sviluppare una libreria unica, che funzioni per il backend in C++, e che sia compatibile con l'applicazione web del frontend.

In tale situazione, è garantita la serializzazione e deserializzazione identica dei messaggi, la gestione identica degli errori e che i test di unità siano validi per l'intero ecosistema.

3.7 Comparativa delle tecnologie

Categoria	Soluzione	Persistenza	Sicurezza
Direct	Port Forwarding	No	Bassa
Tunnel	VPN/SSH	No	Alta
Pub/Sub	MQTT	Limitata	Media
BaaS Proxy	Appwrite	Alta	Alta

Tabella 3.1: Analisi comparativa delle tecnologie

3.8 Conclusione

In conclusione, la soluzione proposta rappresenta un avanzamento rispetto ai metodi tradizionali di comunicazione cross-network. Sfruttando la robustezza di un middleware BaaS e la portabilità del binario WebAssembly, il progetto risolve i problemi di latenza, sicurezza e configurazione di rete in un'unica architettura software modulare e manutenibile.

Capitolo 4

Design e implementazione

4.1 Modello di dominio

Per disaccoppiare la logica applicativa dalla rappresentazione JSON dei dati, il progetto introduce un modulo di dominio centralizzato definito nel file `model.h`. Questo modulo raccoglie tutte le strutture dati utilizzate nello scambio di informazioni tra frontend, proxy e backend.

L'obiettivo è fornire una rappresentazione tipizzata e coerente delle entità del sistema, evitando la manipolazione diretta di oggetti `JsonObject` nei livelli superiori.

4.1.1 Struttura del modulo

Tutte le entità sono definite all'interno del namespace `model`, che rappresenta il punto di accesso unico per il dominio applicativo. Le strutture sono progettate per essere, in primis leggere e facilmente serializzabili, indipendenti dal trasporto di dati e riutilizzabili tra i diversi moduli del sistema.

4.1.2 Entità principali

Il modello definisce diverse entità fondamentali:

- **User**: rappresenta un utente tramite identificatore e email, con metodo `isValid()` per verificare la consistenza dei dati;
- **Channel**: identifica un canale di comunicazione tramite un ID e un nome visibile in UI;
- **Message**: rappresenta l'unità base della comunicazione. Contiene un identificatore del canale, un identificatore del mittente, uno per il messaggio, un timestamp in formato UTC, un payload generico in formato JSON ed un flag `isEcho` per distinguere messaggi generati localmente da quelli ricevuti dal server.
- **SessionInfo**: contiene metadati relativi alla sessione, inclusi tipo di autenticazione e timestamp di creazione e scadenza;
- **ChannelMember**: descrive la presenza di un utente in un canale, includendo stato di attività e informazioni temporali;
- **PersistencePolicy**: definisce le politiche di persistenza (TTL) per messaggi, sessioni e canali inattivi;
- **AppCommConfig**: contiene i parametri di configurazione necessari per connettersi al backend Appwrite.

4.1.3 Serializzazione e parsing

Le entità del modello implementano metodi di conversione bidirezionale tra rappresentazione tipizzata e formato JSON. In particolare `fromJson()` consente di costruire oggetti a partire dai payload ricevuti dal backend mentre `toJson()` permette la serializzazione per l'invio dei dati.

Questo approccio centralizza la logica di parsing, evitando duplicazioni e garantendo coerenza tra i diversi componenti del sistema.

4.1.4 Validazione

Le strutture principali includono metodi di validazione come `isValid()`, utilizzati per verificare la presenza e la correttezza dei campi obbligatori prima dell'utilizzo degli oggetti.

Nel caso di parsing da JSON, se i dati non risultano validi, viene restituita un'istanza vuota. Questo consente ai moduli superiori di gestire esplicitamente i casi di errore senza propagare dati inconsistenti.

4.1.5 Gestione della sessione

Il tipo `SessionInfo` include il metodo `isExpired()`, che permette di verificare in modo immediato la validità temporale della sessione. Il tipo di autenticazione è modellato tramite l'enumerazione `AuthType`, che distingue tra sessioni anonime (`guest`) e autenticazione tramite email.

4.1.6 Integrazione con gli altri componenti

Il modello di dominio è utilizzato trasversalmente dai principali moduli del sistema; l'SDK riceve dati in formato JSON dal backend, il `RecoveryManager` converte tali dati in oggetti `model::Message`, la `RecentMessageCache` memorizza solo messaggi validati ed i livelli applicativi lavorano esclusivamente con strutture tipizzate.

Questa architettura garantisce una separazione chiara tra livello di trasporto e logica applicativa, migliorando robustezza, leggibilità e manutenibilità del codice.

4.2 Appwrite SDK

Il modulo `AppwriteSDK` costituisce il cuore tecnologico della libreria `appcomm`. Questo astrae l'API REST¹ di Appwrite, permettendo al resto della libreria di interagire con il backend senza dover gestire manualmente la complessità del protocollo HTTP o della gestione delle sessioni.

L'architettura del modulo segue i principi di ereditarietà, con lo scopo principale di separare le responsabilità per quello che riguarda la parte che interessa al client, e quella che interessa al server.

La scelta di utilizzare Qt, tramite `QNetworkAccessManager`² è principalmente dovuta alla necessità del frontend di eseguire una build con `WebAssembly`, il quale non supporta librerie di comunicazione come `Curl`. Inoltre l'utilizzo di tale libreria, permette la connessione e la comunicazione tramite `signals` e `slots`, il che risulta consistente e comodo per il resto dell'implementazione e soprattutto per il testing del modulo.

¹<https://appwrite.io/docs/apis/rest>

²<https://doc.qt.io/qt-6/qnetworkaccessmanager.html>

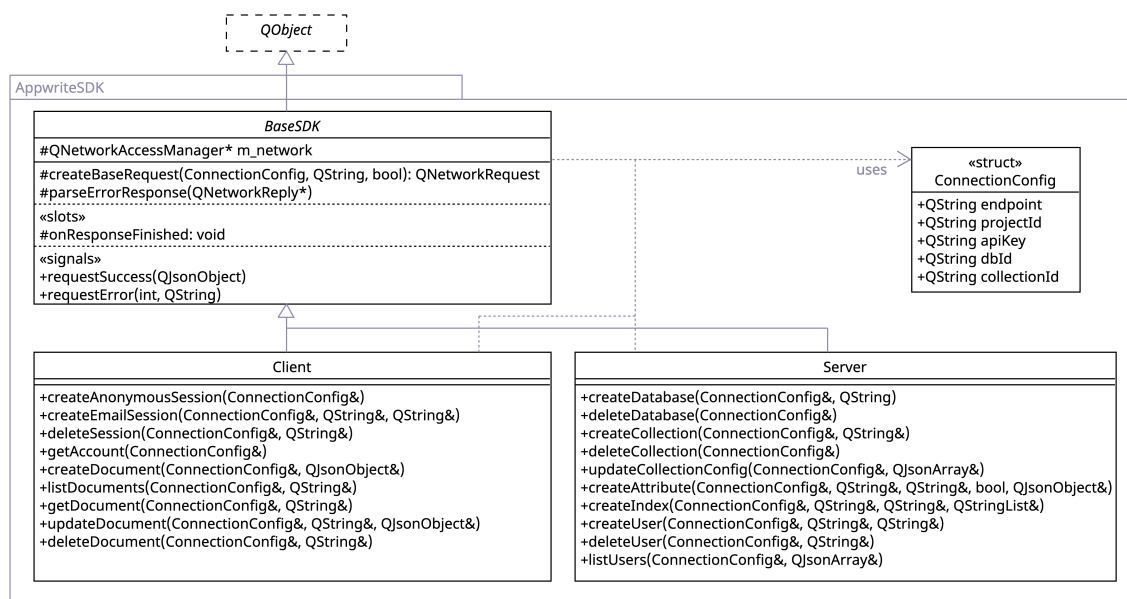


Figura 4.1: Struttura di classi di AppwriteSDK

4.2.1 Componenti principali

1. **ConnectionConfig**: Struttura che incapsula le credenziali e gli identificatori necessari per la comunicazione;
2. **BaseSDK**: Classe base astratta che gestisce l'istanza di `QNetworkAccessManager`, la gestione dei Cookie (con `QNetworkCookieJar`³) e la logica comune di parsing delle risposte e degli errori. Qui vengono impiegati `signals` e `slots` per la comunicazione.
3. **Client**: Specializzazione dedicata al frontend. Gestisce tutte le operazioni non-privilegiate, come l'autenticazione anonima o tramite email e la creazione di documenti nel database.
4. **Server**: Specializzazione privilegiata per il backend. Utilizzando la chiave API esegue le operazioni amministrative, come il provisioning del database, la creazione di attributes e la gestione degli utenti.

4.2.2 Comunicazione asincrona (con signals e slots)

A differenza di un approccio sincrono, quindi bloccante, l'utilizzo del meccanismo di Qt `signals` e `slots` è stato scelto per garantire la latenza minima di rete. Quando viene inviata una richiesta alla REST API, il controllo torna immediatamente all'applicazione; una volta ricevuta una risposta, viene emesso un segnale, tra `requestSuccess` o `requestError`. Tutto ciò è essenziale per WebAssembly, del quale il thread principale non può mai essere bloccato.

4.2.3 Gestione della sessione

L'SDK è configurato per utilizzare un `QNetworkCookieJar`. Nel caso del client, permette ad Appwrite id iniettare un cookie di sessione dopo il login. Qt gestisce automaticamente la persistenza di quest'ultimo nelle richieste successive, simulando lo stato di autenticazione tipico di un browser senza dover gestire manualmente i token JWT nel codice.

³<https://doc.qt.io/qt-6/qnetworkaccessmanager.html>

4.2.4 Dualità Server-Client

Sebbene il codice sia condiviso, l'SDK implementa una distinzione netta tra la modalità ADMIN e USER.

Il server agisce come supervisore; crea l'infrastruttura e gestisce l'integrità dei dati. Il client invece interagisce esclusivamente con i propri canali autorizzati. Questa separazione garantisce che le chiavi API amministrative non vengano mai compilate nel binario esposto pubblicamente.

4.2.5 Utilizzo

Per utilizzare il modulo, è dapprima necessario definire una `ConnectionConfig` e istanziare uno dei due componenti desiderati (`Client/Server`):

```
AppwriteSDK::ConnectionConfig config;
config.endpoint = "https://cloud.appwrite.io/v1";
config.projectId = "my-project-id";
config.dbId = "chat-db";
// Altri campi...

QNetworkAccessManager* mgr = new QNetworkAccessManager(this);
AppwriteSDK::Client* client = new AppwriteSDK::Client(mgr, this);
```

Di seguito un esempio di flusso, dove viene effettuato un login e l'invio di un messaggio:

```
// Connessione ai segnali
connect(client &AppwriteSDK::Client::requestSuccess, this,
[] (const QJsonObject& data) {
    qDebug() << "Success: " << data;
});

// Login Asincrono
client->createEmailSession(config, "user@example.com", "password123");

// Invio messaggio
QJsonObject msg;
msg["payload"] = "Hello, World!";
msg["channelId"] = "room-01";
client->createDocument(config, msg);
```

4.2.6 Gestione degli errori

Incluso nell'SDK, è presente un parser intelligente, che estrae i messaggi di errore restituiti da Appwrite (per esempio *Invalid credentials* o *Document not found*) e li propaga attraverso il segnale `requestError`, permettendo una gestione centralizzata dei guasti nel modulo superiore di logica di business.

4.3 Resilienza e controllo

Oltre alla gestione del trasporto dati, tramite l'SDK, la libreria `appcomm` integra dei componenti specializzati per garantire la continuità del servizio, la protezione delle risorse e la coerenza di stato tra client e server.

Questo garantisce che la logica di business sia separata dal trasporto, permettendo la gestione in modo robusto delle interruzioni di rete e dei limiti infrastrutturali.

4.3.1 Cache dei messaggi

È quindi presente un modulo, `RecentMessageCache`, il quale è responsabile della gestione di un buffer di memoria volatile che conservi i messaggi transitati più di recente. La sua funzione principale è ridurre la latenza d'accesso ai dati recenti e fornire una base per le operazioni quando è necessario recuperare un'eventuale perdita di messaggi.

RecentMessageCache
-int m_capacity -QQueue<QString> m_queue -QHash<QString, Message> m_cache
+addMessage(Message&) +getMessagesSince(const QString&) -> QList<Message> +getMessage(const QString&) -> Message +getAllMessages() -> QList<Message> +contains(const QString&) -> bool +clear() +size() -> int +capacity() -> int +isEmpty() -> bool

Figura 4.2: Struttura di `RecentMessageCache`

Algoritmo di *Eviction*

La cache è implementata seguendo una politica LRU semplificata. Per massimizzare le prestazioni, vengono utilizzate due strutture dati coordinate:

1. `QHash<QString, model::Message>`⁴: `HashMap` che garantisce la ricerca con complessità $O(1)$ per l'estrazione rapida di un singolo messaggio tramite il suo ID.
2. `QQueue<QString>`⁵: mantiene l'ordine cronologico dei messaggi (FIFO).

Quando la cache raggiunge la capacità massima assegnata (`m_capacity`), l'elemento più vecchio viene rimosso (*evicted*) per fare spazio ai nuovi dati, tenendo traccia degli effettivi ultimi n messaggi.

Integrità dei dati e supporto alla resilienza

L'integrazione con `model::Message` permette alla cache di validare ogni inserimento tramite il metodo `msg.isValid()`. Questo assicura che solo dati strutturalmente corretti vengano memorizzati.

Il metodo di gestione `getMessagesSince(QString messageId)` analizza la coda cronologica e restituisce tutti i messaggi successivi ad un determinato ID. Questa funzione è vitale per gestire brevi interruzioni di rete, permettendo al frontend di ricostruire la cronologia mancante, senza dover consultare il server remoto, in quanto possibilmente già ricevuti e presenti nel buffer parziale.

4.3.2 Gestione della recovery

Il modulo `RecoveryManager` agisce come orchestratore per il riallineamento dello stato del client con il backend `Appwrite`. Il suo scopo è colmare lacune informative causate da instabilità della rete o avvii a freddo dell'applicazione.

⁴<https://doc.qt.io/qt-6/qhash.html>

⁵<https://doc.qt.io/qt-6/qqueue.html>

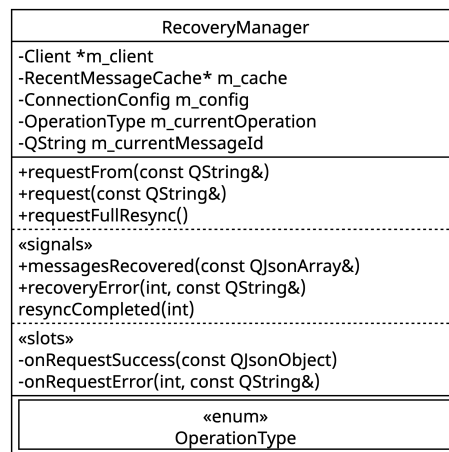


Figura 4.3: Struttura di RecoveryManager

Strategie di recupero e *Gap Detection*

Il modulo gestisce diverse modalità operative per garantire l'integrità dei dati; di seguito una breve descrizione:

1. **Recupero delta (`requestFrom`):** Identifica i messaggi mancanti rispetto all'ultimo ID noto. Se la cache non contiene l'intera sequenza, il manager effettua una query asincrona al server utilizzando i filtri temporali estratti dai metadati del messaggio.
2. **Sincronizzazione globale (`requestFullResync`):** Esegue una query per ottenere gli ultimi n messaggi dalla collezione, garantendo un punto di recovery certo, pensato per scenari di desincronizzazione critica.

Pipeline di trasformazione asincrona

Il manager funge da ponte tra il trasporto JSON ed il modello tipizzato. Quando l'SDK riceve dati dal server, il `RecoveryManager` esegue il parsing tramite `model::Message::fromJson()`, valida i messaggi e popola la cache, prima di emettere il segnale `messagesRecovered` verso il frontend.

4.3.3 Controllo del Throughput

La prevenzione del superamento delle quote API imposte dal server e la protezione dell'infrastruttura dai carichi eccessivi è di vitale importanza per qualsiasi servizio che voglia offrire un'esperienza di utilizzo decente. Per fare ciò è stato implementato un modulo di limitazione del traffico, basato sull'algoritmo *Token Bucket* [4].

Token Bucket [4]

Il sistema gestisce un "secchio" di token, che definisce la capacità massima di richieste simultanee, anche detta *burst size*. Un token viene consumato ad ogni richiesta e questi vengono rigenerati ad un ritmo costante, il quale è presente come stato `m_refillRate`. Se il numero di token disponibili è zero, la richiesta viene negata, quindi effettivamente messa in attesa finché, una volta che i token sono di nuovo disponibili, potrà riprovare; tutto ciò viene svolto tramite il metodo `allowRequest()`.

RateLimiter
-int m_availableTokens -QDateTime m_lastRefill -int m_maxTokens -int m_refillRate
+allowRequest() -> bool +availableTokens() -> int +maxTokens() -> int +refillRate() -> int +reset() -refillTokens()

Figura 4.4: Struttura di RateLimiter

Ottimizzazione delle risorse

L'implementazione adotta una strategia di refill passivo, detta *lazy refill*: invece di utilizzare timer attivi che graverebbero sulla CPU, il calcolo della rigenerazione avviene *on-demand* ogni volta che viene richiesto un accesso, misurando i millisecondi trascorsi dall'ultima operazione riuscita. Questo approccio garantisce un controllo preciso del traffico con un overhead computazionale trascurabile.

4.4 Gestione degli errori

Per gestire in modo uniforme i fallimenti all'interno del sistema, il progetto introduce un modello di errore centralizzato definito nel file `errorModel.h`. Questo modulo consente di rappresentare e propagare gli errori in maniera coerente tra frontend, proxy e backend.

L'obiettivo è evitare l'uso di codici di errore impliciti o messaggi non strutturati, fornendo invece una rappresentazione tipizzata e standardizzata delle condizioni di errore.

4.4.1 Classificazione degli errori

Gli errori sono classificati tramite l'enumerazione `ErrorCode`, che definisce un insieme finito di categorie:

Codice	Descrizione
None	assenza di errore
AuthError	errore di autenticazione
NetworkError	problemi di rete
ParseError	errori durante il parsing dei dati
ProtocolError	comportamento non conforme al protocollo
PermissionError	accesso negato
TimeoutError	timeout della richiesta
RateLimitExceeded	superamento dei limiti di richiesta
Unknown	errore non classificato

Tabella 4.1: Codici di errore

Questa classificazione permette di distinguere rapidamente la natura del problema e di gestirlo in modo appropriato nei diversi livelli dell'applicazione.

4.4.2 Struttura dell'errore

La struttura `AppCommError` rappresenta un errore completo all'interno del sistema. Essa include un codice di errore (`ErrorCode`), un messaggio descrittivo leggibile (`message`) ed un codice di stato HTTP opzionale (`httpStatus`), utile quando l'errore deriva da operazioni REST.

Questa struttura consente di trasportare sia informazioni tecniche sia informazioni utili per il debug o la visualizzazione lato utente.

4.4.3 Rilevazione e formattazione

Il metodo `hasError()` permette di verificare rapidamente se l'oggetto rappresenta un errore, controllando che il codice sia diverso da `ErrorCode::None`. Questo evita l'uso di controlli sparsi o condizioni implicite.

La funzione `errorCodeToString()` fornisce una rappresentazione testuale del codice di errore, utile per logging e debugging.

Il metodo `toString()` restituisce invece una stringa formattata che include il tipo di errore, il codice HTTP associato ed il messaggio descrittivo.

Ad esempio, un errore può essere rappresentato come:

```
Error[NetworkError] (500): Internal server error
```

4.4.4 Integrazione nel sistema

Il modello di errore è utilizzato trasversalmente nei diversi componenti del sistema; Mentre l'SDK segnala errori di rete o autenticazione, il parsing dei dati invece lo utilizza `ParseError` in caso di payload non validi, il proxy può generare errori di protocollo o permessi ed i moduli applicativi possono reagire in base al tipo di errore ricevuto.

L'uso di un modello centralizzato consente di evitare ambiguità e di uniformare la gestione degli errori in tutto il sistema, migliorando la robustezza e la leggibilità del codice.

4.4.5 Scelte progettuali

L'introduzione di un tipo dedicato per gli errori evita la propagazione di stringhe non strutturate o codici numerici non documentati. Questo approccio rende esplicito il significato degli errori e facilita l'estensione futura del sistema con nuove categorie o metadati aggiuntivi.

Inoltre, la separazione tra codice di errore, messaggio e stato HTTP permette di gestire in modo flessibile sia errori interni sia errori derivanti da servizi esterni.

4.5 Gestione dello stato del client

Per gestire il comportamento dinamico del client durante l'esecuzione, il progetto introduce un modello di stato definito nel file `state.h`. Questo modulo rappresenta lo stato runtime del client, includendo informazioni sulla connessione, sulla sessione e sul contesto di comunicazione attivo.

L'obiettivo è fornire una rappresentazione centralizzata dello stato del client, che possa essere condivisa tra i diversi componenti e utilizzata per sincronizzare logica applicativa e interfaccia utente.

4.5.1 Struttura dello stato

Lo stato del client è rappresentato dalla struttura `ClientState`, che contiene le informazioni necessarie per descrivere il contesto corrente della comunicazione. In particolare, essa include:

- **lastReceivedSequence**: numero di sequenza dell'ultimo messaggio ricevuto, utilizzato per garantire l'ordine dei messaggi e rilevare eventuali gap nella comunicazione, impostato di default a -1 per indicare che nessun messaggio è stato ancora ricevuto;
- **lastReceivedMessageId**: identificatore dell'ultimo messaggio ricevuto, utile per operazioni di recovery o deduplicazione;
- **activeChannelId**: identificatore del canale attualmente attivo, che definisce il contesto della comunicazione;
- **authenticated**: flag che indica se il client ha completato con successo il processo di autenticazione, impostato di default a false per indicare che l'utente non è autenticato.

Questa struttura rappresenta una fotografia dello stato corrente del client e viene aggiornata dinamicamente durante l'esecuzione.

4.5.2 Gestione dell'ordinamento e recovery

I campi `lastReceivedSequence` e `lastReceivedMessageId` sono fondamentali per garantire la correttezza del flusso dei messaggi. Il numero di sequenza permette di verificare l'ordine dei messaggi ricevuti, individuare eventuali messaggi mancanti (gap) e di supportare meccanismi di recovery basati su sequenze.

L'identificatore del messaggio, invece, può essere utilizzato per prevenire duplicazioni e per mantenere coerenza tra client e backend.

4.5.3 Contesto di comunicazione

Il campo `activeChannelId` definisce il contesto corrente in cui il client opera. Questo consente di filtrare i messaggi rilevanti e di limitare le operazioni al canale attivo.

Separare esplicitamente il canale attivo evita ambiguità e semplifica la gestione della comunicazione in presenza di più canali.

4.5.4 Stato di autenticazione

Il flag `authenticated` rappresenta lo stato di autenticazione del client. Questo valore è utilizzato per abilitare o disabilitare operazioni che richiedono una sessione valida.

In particolare, consente di impedire l'invio o la ricezione di messaggi quando il client non è autenticato, garantendo coerenza con le politiche di sicurezza del sistema.

4.5.5 Stato della connessione

Lo stato della connessione è rappresentato dall'enumerazione `ConnectionState`, che descrive il ciclo di vita della connessione tra client e backend.

Le possibili condizioni includono:

Questa rappresentazione esplicita permette di gestire in modo chiaro il comportamento del sistema in funzione dello stato della rete.

Codice	Descrizione
Disconnected	assenza di errore
Connecting	errore di autenticazione
Connected	connessione attiva e funzionante;
Reconnecting	tentativo di ripristino dopo una disconnessione;
Disconnecting	chiusura della connessione in corso;
Failed	errore durante il tentativo di connessione.

Tabella 4.2: Stati di connessione

4.5.6 Integrazione con gli altri componenti

Il modello di stato è utilizzato da diversi moduli del sistema:

- il `MessageProcessor` utilizza `lastReceivedSequence` per verificare duplicati e gap;
- il `RecoveryManager` sfrutta le informazioni di stato per riallineare il client con il backend;
- i componenti di rete aggiornano lo stato della connessione;
- l'interfaccia utente può reagire ai cambiamenti di stato per abilitare o disabilitare funzionalità.

Questo approccio consente di mantenere una visione coerente e sincronizzata dello stato del sistema, migliorando la robustezza e la prevedibilità del comportamento del client.

4.5.7 Scelte progettuali

La separazione tra stato del client e logica operativa consente di ridurre l'accoppiamento tra i componenti e di semplificare la gestione della complessità.

Centralizzare lo stato in una struttura dedicata permette inoltre di facilitare il debugging, rendere esplicite le dipendenze tra componenti e supportare estensioni future del sistema senza modifiche invasive.

4.6 Elaborazione dei messaggi in ingresso

Per gestire in modo controllato i messaggi ricevuti dal backend, il client introduce il componente `MessageProcessor`. Questa classe ha il compito di analizzare i messaggi in ingresso prima che vengano propagati alla façade `Client` dell'applicazione.

L'obiettivo è garantire che tutti i messaggi validi, coerenti con il contesto corrente e non duplicati vengano accettati dal client. In questo modo, il componente contribuisce a mantenere la correttezza del flusso di comunicazione e a supportare i meccanismi di recovery (vedi 4.3.2).

4.6.1 Ruolo del componente

La classe `MessageProcessor` opera come filtro logico sui messaggi ricevuti. In particolare, essa verifica la validità strutturale del messaggio, la coerenza con il canale attualmente attivo, l'assenza di duplicati e la presenza di eventuali gap nella sequenza dei messaggi.

Solo i messaggi che superano tutti questi controlli vengono restituiti come validi al chiamante.

4.6.2 Dipendenza dallo stato del client

Il processore mantiene un riferimento a un oggetto `ClientState`, da cui ricava il contesto runtime necessario per prendere decisioni sui messaggi ricevuti. In particolare, vengono utilizzati:

- `activeChannelId`, per verificare che il messaggio appartenga al canale attualmente selezionato;
- `lastReceivedSequence`, per controllare l'ordine dei messaggi e individuare duplicati o gap.

Questo rende `MessageProcessor` strettamente integrato con il modello di stato del client, senza però incorporare direttamente la logica di connessione o di recovery.

4.6.3 Metodo `processIncoming()`

Il metodo principale della classe è `processIncoming()`, che riceve un oggetto `model::Message` e restituisce un valore di tipo `std::optional<model::Message>`.

Questa scelta progettuale permette di esprimere in modo esplicito due possibili esiti:

- presenza di un messaggio valido, che può essere elaborato dai moduli successivi;
- assenza di risultato, nel caso in cui il messaggio venga scartato.

L'elaborazione avviene verificando la validità del messaggio tramite `message.isValid()`, successivamente controllando il canale attivo, scartando i messaggi che non appartengono al contesto corrente, poi rilevando i duplicati tramite `isDuplicate()`, dopo di che rilevando eventuali gap tramite `hasGap()` e infine aggiornando lo stato del client con l'ultima sequenza ricevuta.

Se uno dei controlli fallisce, il metodo restituisce `std::nullopt`. In caso contrario, restituisce il messaggio originale.

4.6.4 Deduplicazione

La deduplicazione è affidata al metodo `isDuplicate()`, che confronta il numero di sequenza del messaggio in ingresso con il valore di `lastReceivedSequence` memorizzato nello stato del client.

Un messaggio viene considerato duplicato se la sua sequenza è minore o uguale all'ultima sequenza già accettata. Questa scelta impedisce che messaggi ricevuti più volte vengano propagati nuovamente all'applicazione.

4.6.5 Rilevazione di gap

Il metodo `hasGap()` verifica se esiste un salto nella sequenza dei messaggi ricevuti. Se non è ancora stato ricevuto alcun messaggio, cioè se `lastReceivedSequence` è inizializzato a `-1`, il metodo restituisce `false`.

Negli altri casi, viene rilevato un gap quando la sequenza del nuovo messaggio è maggiore di `lastReceivedSequence + 1`. Questo segnala la possibile perdita di uno o più messaggi e costituisce il punto di aggancio per i meccanismi di recovery.

4.6.6 Aggiornamento dello stato

Quando un messaggio viene accettato, il processore aggiorna il campo `lastReceivedSequence` nello stato del client. In questo modo, i messaggi successivi possono essere confrontati rispetto all'ultima sequenza valida già elaborata.

Questo aggiornamento centralizzato garantisce coerenza tra il flusso dei messaggi ricevuti e lo stato runtime mantenuto dal client.

4.6.7 Reset del contesto

Il metodo `reset()` consente di reinizializzare il processore rispetto a un nuovo canale attivo. In particolare, esso:

- aggiorna il valore di `activeChannelId`;
- reimposta `lastReceivedSequence` a -1.

Questa operazione è necessaria quando il client cambia canale, poiché la sequenza dei messaggi deve essere considerata nuovamente a partire da uno stato iniziale.

4.6.8 Scelte progettuali

La classe `MessageProcessor` separa la logica di validazione del flusso dei messaggi dalla logica di trasporto e dalla logica applicativa. Questo riduce l'accoppiamento tra i componenti e rende più semplice testare in modo isolato i casi di duplicazione, disallineamento e filtro per canale.

L'uso combinato di `ClientState`, `model::Message` e `std::optional` rende inoltre il comportamento del componente esplicito, prevedibile e facilmente estendibile con future strategie di recovery.

Capitolo 5

Test e validazione

5.1 Risultati

Capitolo 6

Risultati

6.1 Manuale d'uso

Capitolo 7

Conclusioni

7.1 Sviluppi futuri

Glossario

API Application Programming Interface. Un insieme di definizioni e protocolli utilizzati per costruire e integrare il software applicativo. 1

Appwrite Una piattaforma open-source per lo sviluppo di applicazioni web e mobile, che fornisce una serie di servizi backend come autenticazione, database, storage e funzioni serverless. 4

FIFO First In First Out, modalità di gestione di una coda di dati, nella quale i primi elementi aggiunti sono anche i primi ad essere rimossi. 14

Inbound Comunicazione di rete direzionata dall'esterno verso la parte interna di una rete LAN. 7

JSON JavaScript Object Notation. Un formato di testo standard aperto, leggero e leggibile dall'uomo, utilizzato per lo scambio di dati tra client e server. 1

LRU Least Recently Used, modalità di eviction di una memoria volatile, dove, una volta raggiunta la capacità massima, il dato aggiunto meno di recente viene espulso, in favore di uno nuovo. 14

NAT Network Address Translation. Una tecnica usata dai router per modificare gli indirizzi IP dei pacchetti in transito, permettendo a più dispositivi di condividere un singolo indirizzo IP pubblico. 1, 7

Outbound Comunicazione di rete direzionata verso l'esterno di una rete locale LAN. 7

Proxy Un componente software o hardware che funge da intermediario tra un client e un server, inoltrando le richieste e le relative risposte. 1

Throughput Capacità di trasmissione effettiva che viene usata in una certa unità di tempo. i, 15

TTL Time to Live, intervallo di tempo della durata di validità di un dato prima della sua scadenza o eliminazione automatica. È solitamente espresso in secondi ed è utilizzato per gestire la persistenza temporanea di elementi come messaggi, sessioni o risorse di cache. 10

UUID Universally Unique Identifier. Un identificativo standardizzato a 128 bit utilizzato nei sistemi informatici per identificare in modo univoco informazioni e risorse. 1

WebAssembly Un formato di istruzioni binario portatile, progettato per consentire l'esecuzione di codice ad alte prestazioni all'interno dei browser web. 1, 4

Sitografia

- [1] Appwrite Team. *Appwrite Documentation - Open-Source Backend Server*. 2026. URL: <https://appwrite.io/docs> (visitato il 18/02/2026).
- [2] The Qt Company. *Qt 6 Documentation*. 2026. URL: <https://doc.qt.io/qt-6/> (visitato il 18/02/2026).
- [3] WebAssembly Community Group. *WebAssembly Specification*. 2026. URL: <https://webassembly.org/> (visitato il 18/02/2026).
- [4] Geeks for Geeks. *Token Bucket Algorithm*. 2026. URL: <https://geeksforgeeks.org/computer-networks/token-bucket-algorithm/> (visitato il 16/03/2026).